Strings

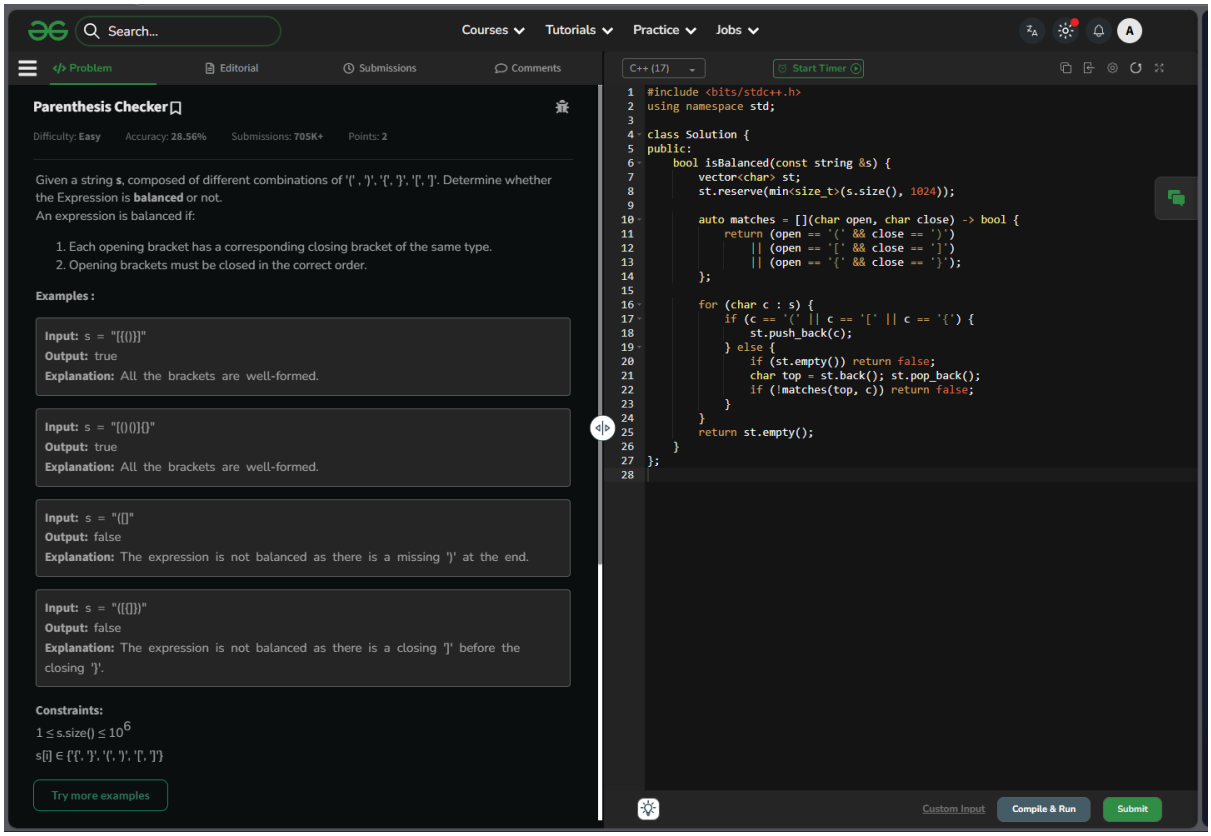
Name: Arjun Mankuskar

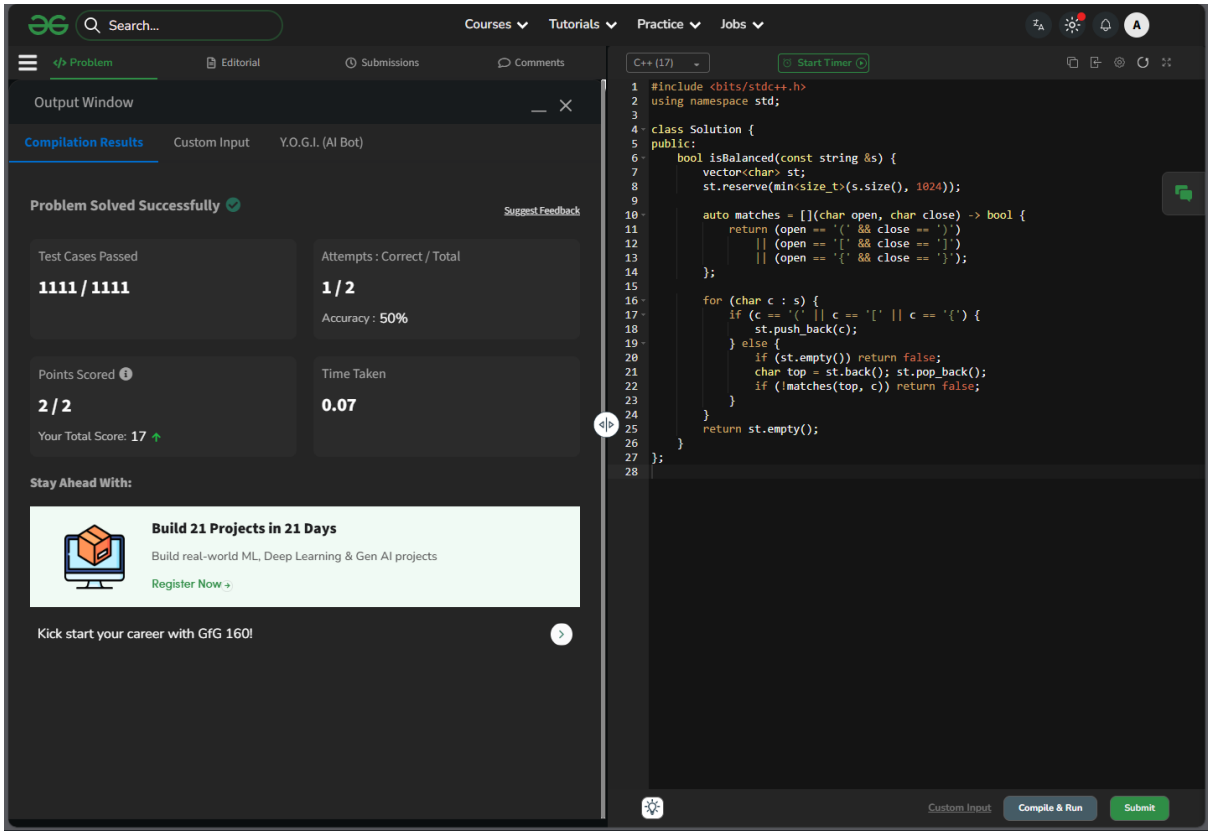
PRN:202401110038

Branch: CSE (AIML)

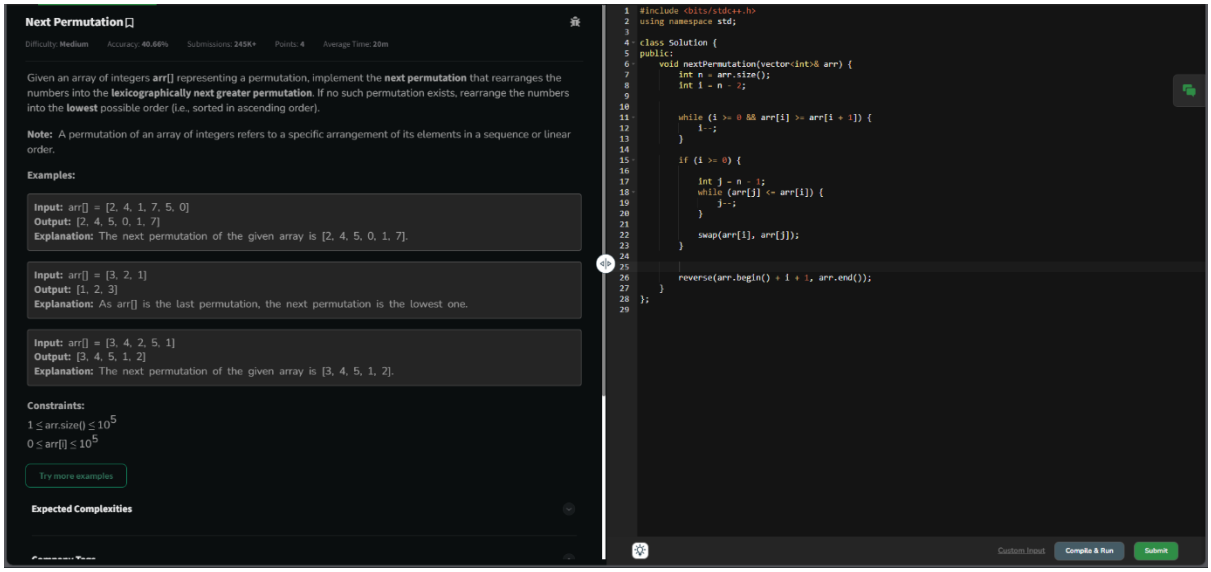
Batch: A2

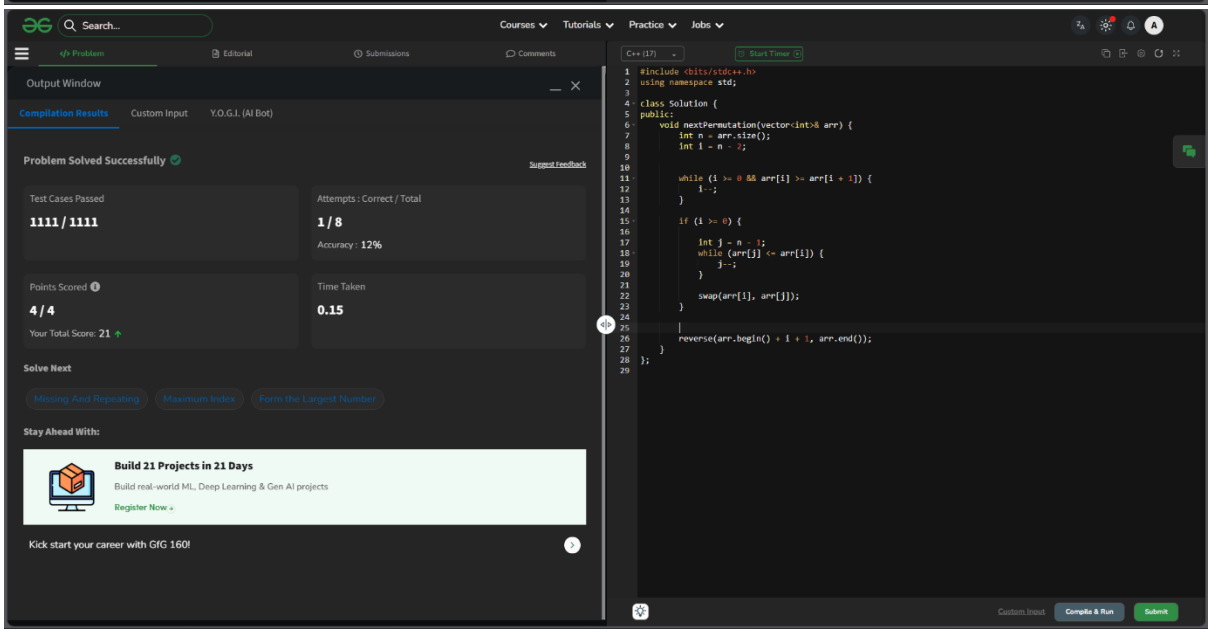
1.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Parenthesis Checker
Difficulty: Easy
Accuracy: 28.56%
Submissions: 705K+
Points: 2
Given a string s, composed of different combinations of '(' , ')', '{', '}', '[', ']'. Determine whether the Expression is balanced or not.
An expression is balanced if:
1. Each opening bracket has a corresponding closing bracket of the same type.
2. Opening brackets must be closed in the correct order.
Examples :
Input: s = "[{()}]"
Output: true
Explanation: All the brackets are well-formed.
Input: s = "[()()]{}"
Output: true
Explanation: All the brackets are well-formed.
Input: s = "([]"
Output: false
Explanation: The expression is not balanced as there is a missing ')' at the end.
Input: s = "([{}])"
Output: false
Explanation: The expression is not balanced as there is a closing ']' before the closing ')'.
Constraints:
1 ≤ s.size() ≤ 10^6
s[i] ∈ {'{', '}', '(', ')', '[', ']'}
Try more examples
C++ (17)
Start Timer
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
bool isBalanced(const string &s) {
vector<char> st;
st.reserve(min<size_t>(s.size(), 1024));
auto matches = [](char open, char close) -> bool {
return (open == '(' && close == ')')
|| (open == '[' && close == ']')
|| (open == '{' && close == '}');
};
for (char c : s) {
if (c == '(' || c == '[' || c == '{') {
st.push_back(c);
} else {
if (st.empty()) return false;
char top = st.back(); st.pop_back();
if (!matches(top, c)) return false;
}
}
return st.empty();
}
};
Custom Input
Compile & Run
Submit

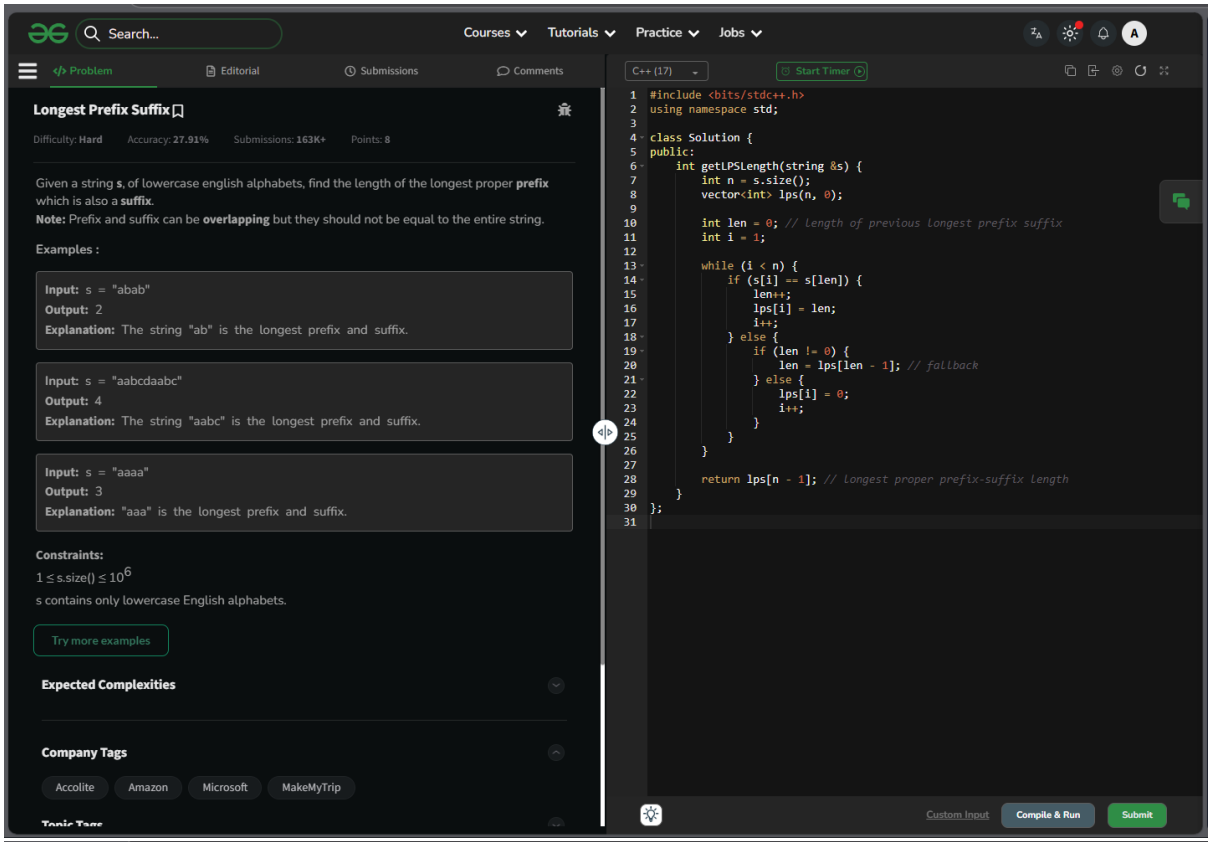

Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Problem Solved Successfully
Suggest Feedback
Test Cases Passed
1111 / 1111
Attempts : Correct / Total
1 / 2
Accuracy : 50%
Points Scored
2 / 2
Your Total Score: 17
Time Taken
0.07
Stay Ahead With:
Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects
Register Now
Kick start your career with GfG 160!
C++ (17)
Start Timer
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
bool isBalanced(const string &s) {
vector<char> st;
st.reserve(min<size_t>(s.size(), 1024));
auto matches = [](char open, char close) -> bool {
return (open == '(' && close == ')')
|| (open == '[' && close == ']')
|| (open == '{' && close == '}');
};
for (char c : s) {
if (c == '(' || c == '[' || c == '{') {
st.push_back(c);
} else {
if (st.empty()) return false;
char top = st.back(); st.pop_back();
if (!matches(top, c)) return false;
}
}
return st.empty();
}
};
Custom Input
Compile & Run
Submit

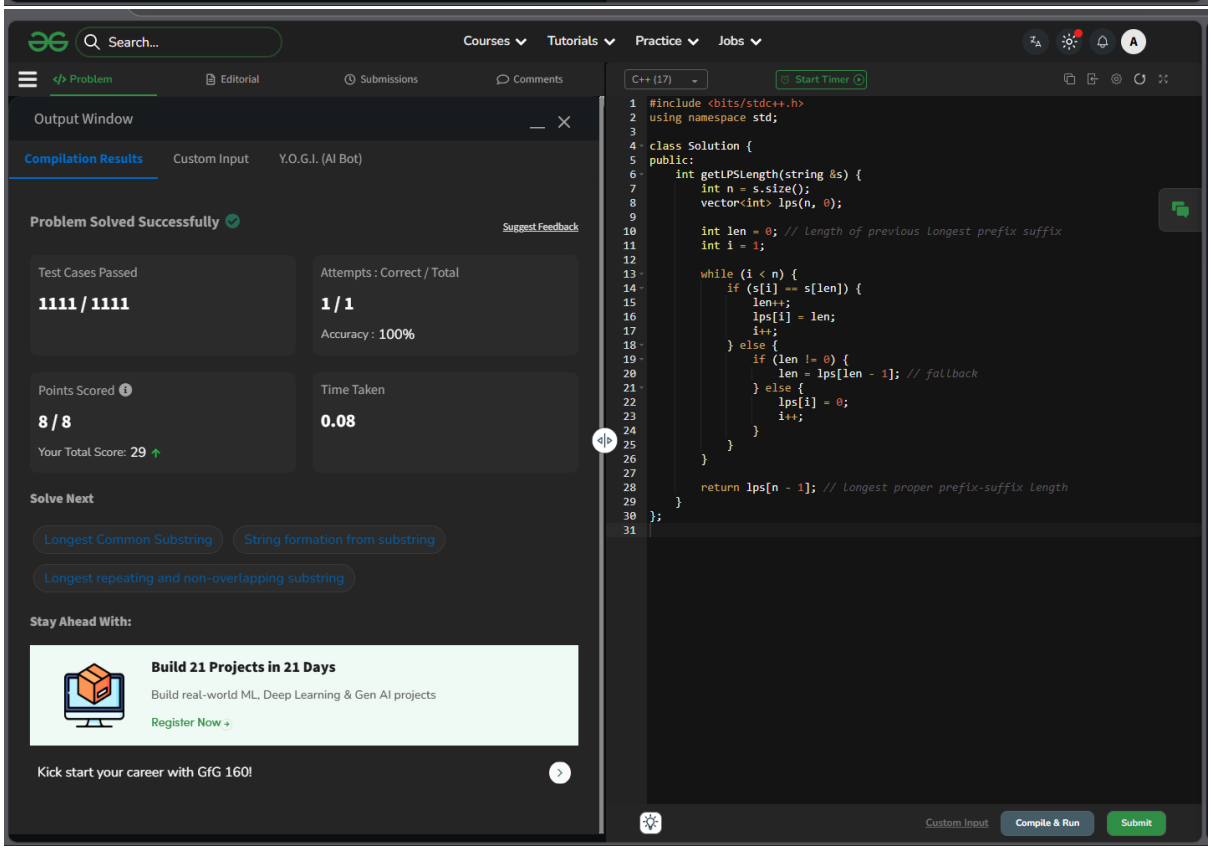
2.


Next Permutation
Difficulty: Medium Accuracy: 40.66% Submissions: 245K+ Points: 4 Average Time: 20m
Given an array of integers arr[] representing a permutation, implement the next permutation that rearranges the numbers into the lexicographically next greater permutation. If no such permutation exists, rearrange the numbers into the lowest possible order (i.e., sorted in ascending order).
Note: A permutation of an array of integers refers to a specific arrangement of its elements in a sequence or linear order.
Examples:
Input: arr[] = [2, 4, 1, 7, 5, 0]
Output: [2, 4, 5, 0, 1, 7]
Explanation: The next permutation of the given array is [2, 4, 5, 0, 1, 7].
Input: arr[] = [3, 2, 1]
Output: [1, 2, 3]
Explanation: As arr[] is the last permutation, the next permutation is the lowest one.
Input: arr[] = [3, 4, 2, 5, 1]
Output: [3, 4, 5, 1, 2]
Explanation: The next permutation of the given array is [3, 4, 5, 1, 2].
Constraints:
1 ≤ arr.size() ≤ 10^5
0 ≤ arr[i] ≤ 10^5
Try more examples
Expected Complexities
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
void nextPermutation(vector<int>& arr) {
int n = arr.size();
int i = n - 2;
while (i >= 0 && arr[i] >= arr[i + 1]) {
i--;
}
if (i >= 0) {
int j = n - 1;
while (arr[j] <= arr[i]) {
j--;
}
swap(arr[i], arr[j]);
}
reverse(arr.begin() + i + 1, arr.end());
}
};
Custom Input
Compile & Run
Submit


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Problem Solved Successfully
Suggest Feedback
Test Cases Passed
1111 / 1111
Attempts : Correct / Total
1 / 8
Accuracy : 12%
Points Scored
4 / 4
Your Total Score: 21
Time Taken
0.15
Solve Next
Missing And Repeating
Maximum Index
Form the Largest Number
Stay Ahead With:
Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects
Register Now
Kick start your career with GfG 160!
C++ (17)
Start Timer
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
void nextPermutation(vector<int>& arr) {
int n = arr.size();
int i = n - 2;
while (i >= 0 && arr[i] >= arr[i + 1]) {
i--;
}
if (i >= 0) {
int j = n - 1;
while (arr[j] <= arr[i]) {
j--;
}
swap(arr[i], arr[j]);
}
reverse(arr.begin() + i + 1, arr.end());
}
};
Custom Input
Compile & Run
Submit

3.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Longest Prefix Suffix
Difficulty: Hard
Accuracy: 27.91%
Submissions: 163K+
Points: 8
Given a string s, of lowercase english alphabets, find the length of the longest proper prefix which is also a suffix.
Note: Prefix and suffix can be overlapping but they should not be equal to the entire string.
Examples :
Input: s = "abab"
Output: 2
Explanation: The string "ab" is the longest prefix and suffix.
Input: s = "aabcdaabc"
Output: 4
Explanation: The string "aabc" is the longest prefix and suffix.
Input: s = "aaaa"
Output: 3
Explanation: "aaa" is the longest prefix and suffix.
Constraints:
1 ≤ s.size() ≤ 10^6
s contains only lowercase English alphabets.
Try more examples
Expected Complexities
Company Tags
Accolite
Amazon
Microsoft
MakeMyTrip
C++ (17)
Start Timer
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
int getLPSLength(string &s) {
int n = s.size();
vector<int> lps(n, 0);
int len = 0; // Length of previous longest prefix suffix
int i = 1;
while (i < n) {
if (s[i] == s[len]) {
len++;
lps[i] = len;
i++;
} else {
if (len != 0) {
len = lps[len - 1]; // fallback
} else {
lps[i] = 0;
i++;
}
}
}
return lps[n - 1]; // Longest proper prefix-suffix length
}
};
Custom Input
Compile & Run
Submit


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Problem Solved Successfully
Suggest Feedback
Test Cases Passed
1111 / 1111
Attempts : Correct / Total
1 / 1
Accuracy : 100%
Points Scored
8 / 8
Your Total Score: 29
Time Taken
0.08
Solve Next
Longest Common Substring
String formation from substring
Longest repeating and non-overlapping substring
Stay Ahead With:
Build 21 Projects in 21 Days
Build real-world ML, Deep Learning & Gen AI projects
Register Now
Kick start your career with GfG 160!
C++ (17)
Start Timer
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
int getLPSLength(string &s) {
int n = s.size();
vector<int> lps(n, 0);
int len = 0; // Length of previous longest prefix suffix
int i = 1;
while (i < n) {
if (s[i] == s[len]) {
len++;
lps[i] = len;
i++;
} else {
if (len != 0) {
len = lps[len - 1]; // fallback
} else {
lps[i] = 0;
i++;
}
}
}
return lps[n - 1]; // Longest proper prefix-suffix length
}
};
Custom Input
Compile & Run
Submit